第 51 届 ICPC 国际大学生程序设计竞赛亚洲区域赛（西安）题解

Coffee_zzz, dXqwq, luqyou, MnZn, orangejuice

LLM BENCHMARK

验题规则

- 所有模型使用**最高档 effort**。
- Token 使用量**不设上限**。
- Prompt 鼓励使用 **subagent 并发**。
- 可以获得**提交反馈**。
- 限时 **5 小时**。

参与本次 LLM 验题的选手有……

- GPT 6 Astra Max & Codex
- Gemini 4 Argon High & Antigravity
- Deepseek V4.1 Flash Max & DSH
- Claude 5.5 Opus Max & Claude Code
- Doubao 2.1 Pro Max & Codex

由于比赛可能会上 Universal Cup，

请勿外传本题解

	模型		工具	A	B	C	D	E	F	G	H	I	J	K	L	M	通过题数
	GPT 6 Astra	+		●	●	●	●	●	●	●	●	●	●	●	●	●	**13** / 13
	Gemini 4 Argon	+		●	●	●	●	●	●	●	●	●	●	●	●	●	**13** / 13
	Deepseek V4.1 Flash	+		●	●	●	●	●	●	●	●	●	●	●	●	●	**12** / 13
	Claude Opus 5.5	+		●	●	●	●	●	●	●	●	●	●	●	●	●	**12** / 13
	Doubao 2.1 Pro	+		●	●	●	●	●	●	●	●	●	●	●	●	●	**4** / 13

PROBLEM

Melodiniq

四个人正在解禁一首音游曲目。

每个人初始有一定的血量，打出不太好的判定会扣除一定的血量，血量扣到 ≤ 0 会强制判定**死亡**，每个**未死亡**的玩家连续打出 200 个最好的判定会给剩余**未死亡**的玩家增加一定的血量。

问整首歌结束后四个人的血量。

Constraints

事件的数量不超过 10^5。

Idea by **dXqwq**

Prepared by **dXqwq**

418 Teams Attempted

400 Teams Solved

First Blood: 阿卡纳伊甸 @ 8 min

Expected Difficulty: Easy

Judge Solution: 685 Bytes

SOLUTION

Melodiniq

按照题意模拟即可。

对于 200 Combo，最简单的写法是对于每个人维护一个 Combo 计数器，但暴力往回枚举的做法也可以通过。

时间复杂度 $O(n)$，需要注意题目中已经明确了每个事件会先结算所有人的血量，再判定每个人是否存活，可能会导致一位玩家即使打出了糟糕的判定，也被其他玩家“救活”了。

PROBLEM

Journey

给定一个 $1 \sim n$ 各出现两遍的序列。

构造一个合法括号串，使得连接每对括号串对应的数字后能得到一个环。

Constraints

可以证明一定有解，$n \leq 2 \cdot 10^5$。

Idea by **dXqwq**

Prepared by **dXqwq**

357 Teams Attempted

274 Teams Solved

First Blood: 击中月亮 @ 25 min

Expected Difficulty: Basic

Judge Solution: 514 Bytes

SOLUTION

Journey

对于括号串的首个字符 s_1，考察它的匹配，有两种简单的情况：匹配 s_2 或匹配 s_{2n}。这对应了两种情况：

- (.....)
- ().....

注意到 a_1 不可能同时等于 a_2 和 a_{2n}，我们取其中不相等的一个选择。

对于余下的序列，我们会发现这两个剩余的值都只出现了一次；而对于图，我们会发现这两个点之间已经连接了一条边。

注意到剩余的部分仍然只需要构造合法括号串，考虑归纳地解决这个问题。

我们发现只需将这两个只出现一次的值合并，问题就可以归纳到将剩余的结点连成环，问题得解。

边界的情况是 $n = 1$，此时连成自环即可。

SOLUTION

Journey

该算法在样例上的结果：

连边依次为

$(3,6),(1,7),(5,6),(2,5)$

$,(2,4),(3,1),(4,7)$

Input	3	6	7	1	5	6	2	2	4	3	4	7	1	5
3+6 -> A	(	)	7	1	5	A	2	2	4	A	4	7	1	5
7+1 -> B			(	)	5	A	2	2	4	A	4	B	B	5
5+A -> C					(	)	2	2	4	C	4	B	B	C
2+C -> D							(	D	4	D	4	B	B	)
D+4 -> E								(	)	E	E	B	B	
E+B -> F										(	F	F	)	
Answer	(	)	(	)	(	)	(	(	)	(	(	)	)	)

PROBLEM

Cryptarithm

你需要用完成一段序列的编辑，可以用三种操作：

- 花费 x，在序列后增加一段给定的序列 A。
- 花费 y，给某个位置 $+1$ 或者 -1。
- 花费 z，删除某个位置。

问得到序列 B 的最小代价。

Constraints

$n, m \leq 3000$，$0 \leq a_i \leq 10^9$。

Idea by **dXqwq**

Prepared by **dXqwq**

285 Teams Attempted

253 Teams Solved

First Blood: 击中月亮 @ 8 min

Expected Difficulty: Basic

Judge Solution: 742 Bytes

SOLUTION

Cryptarithm

注意到由于序列长度一定为 $|B|$，所以第一种操作的次数和第三种操作的次数是绑定的。

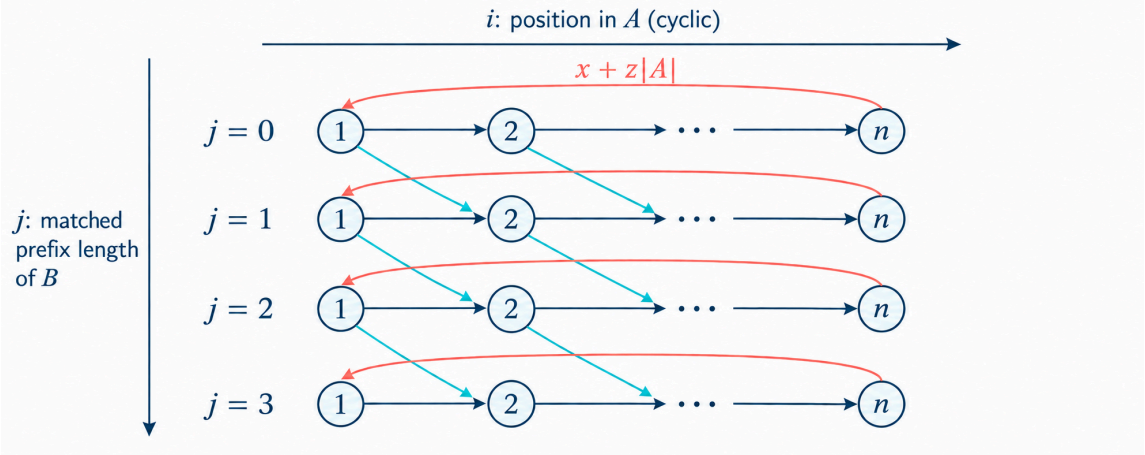
不妨考虑固定第一种操作的次数 d，则问题变为选择一个 A^d 长度为 $|B|$ 的子序列 A'，最小化

$$\sum |A'_i - B_i|$$

并对每个 d 加上第一种操作的代价后取 $\min$。

我们可以用和 LCS 相同方法的 n^2 DP 来解决这个问题，但由于我们需要枚举 d，时间复杂度会到达 $O(n^3)$，无法通过。

一个简单的优化是不显式地枚举 d，我们将这个 DP 中对应 A 的下标改为 $\bmod n$ 循环，只在 $n \to 1$ 的转移中多支付 $x + z \cdot |A|$ 的代价。



此时转移图不再是 DAG，而是一张分层图，由于每层都是环，可以对环循环更新两次来达到 $O(|A| \cdot |B|)$ 的时间复杂度，直接运行 Dijkstra 也可以通过。

PROBLEM

Hydra

你现在有 0 点护甲，需要在第 a_i 个回合获得 $\geq b_i$ 点护甲，可以在每回合前进行任意次以下操作：

- 花费 x，获得一层**覆甲**。
- 花费 y，获得一层**金属化**。

你每回合会获得 $u+v$ 点护甲，其中 u 为覆甲的层数，v 为金属化的层数。每回合结束时，所有护甲会被清空，且你的**覆甲**层数（如果有）会减小 1。

问满足限制的最小代价。

Constraints

$n \leq 5 \cdot 10^5$，$1 \leq x, y, a_i, b_i \leq 10^9$。

Idea by **Coffee_zzz**

Prepared by **Coffee_zzz**

212 Teams Attempted

116 Teams Solved

First Blood: 耳畔虹蓝梦 @ 38 min

Expected Difficulty: Basic

Judge Solution: 632 Bytes

SOLUTION

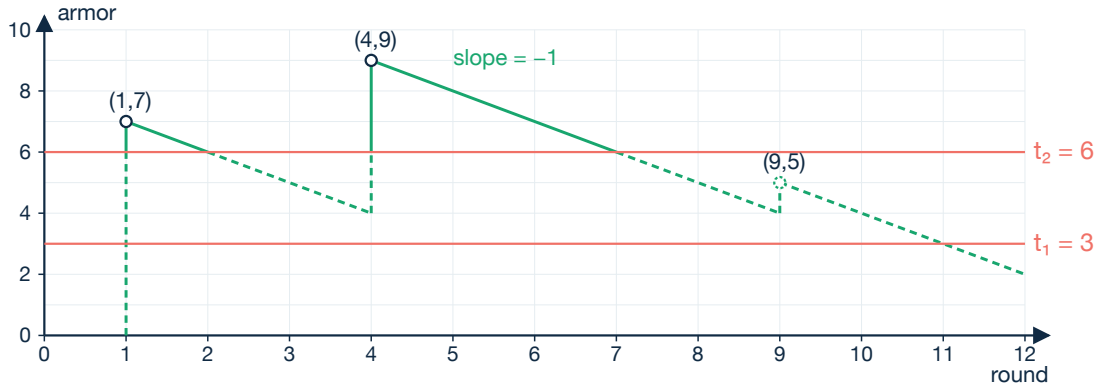
Hydra

枚举获得**金属化**的层数 t，会得到**覆甲**对每轮的护甲需求量。对于这个子问题，考虑如下贪心算法：

- 从小到大考虑 a_i，如果第 a_i 轮剩余的覆甲数量不足 $b_i - t$，则不断获得**覆甲**直到 $b_i - t$。

我们不难证明该算法的正确性。具体地，可以考虑归纳：我们永远会得到一个每一次**覆甲**都最晚获得的方案，而该子结构是不劣的。

接下来考虑优化该算法。我们尝试画出护甲来源的函数：红色部分来自**金属化**，绿色部分来自**覆甲**。



图中绿色竖线部分长度即为**覆甲**获得的次数。

因此可以考虑扫描线维护红色水平线上升时会使得多少条绿色竖线变短，不难发现答案一定在某个交点取到，时间复杂度为按照高度排序的 $O(n \log n)$。

PROBLEM

Gungnir Fracture

给定一个包含 a_i 个 i 的可重集，保证所有数 $< 2^n$。

每次可以选择一组 (m, k) 并删除 $(0 \oplus k, 1 \oplus k, \cdots, m \oplus k)$，问至少几次可以删空。

Constraints

$n \leq 19$，$0 \leq a_i \leq 10^6$。

Idea by **dXqwq**

Prepared by **dXqwq**

44 Teams Attempted

7 Teams Solved

First Blood: 常常一等 @ 130 min

Expected Difficulty: Advanced

Judge Solution: 540 Bytes

SOLUTION

Gungnir Fracture

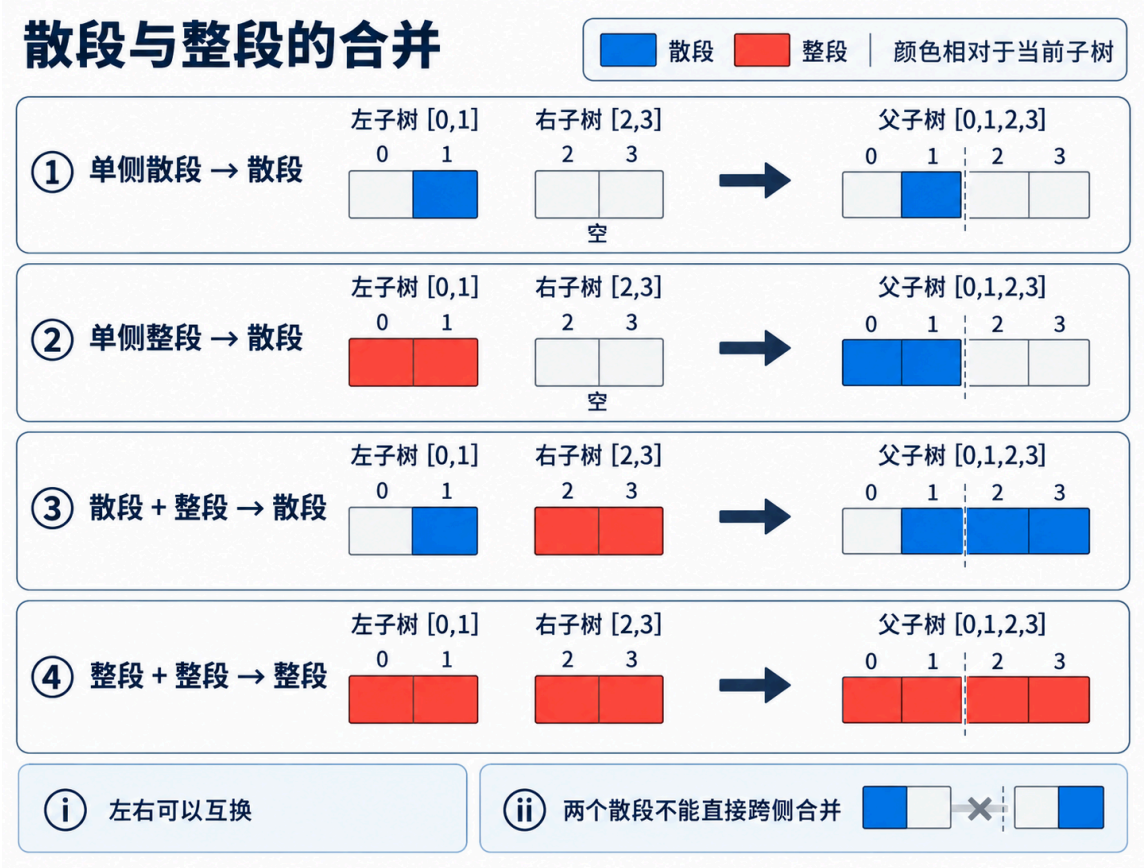
注意到连续段在 XOR 下的结构对称性，容易想到在 01-Trie 上分治解决该问题。

考察一棵子树内的方案，一定是若干个散段和若干个整段组成的。我们考虑该子结构是否能支持合并。

合并时，我们可以做如下操作：

- 两侧的散段/整段 ⇒ 全局散段
- 一侧散段，另一侧整段 ⇒ 全局散段
- 两侧的整段 ⇒ 全局整段

如果你是原教旨选手，可以证明两者的段数可以完全描述子结构，因此如果我们记录所有可能子结构，就可以保证贪心的正确性。

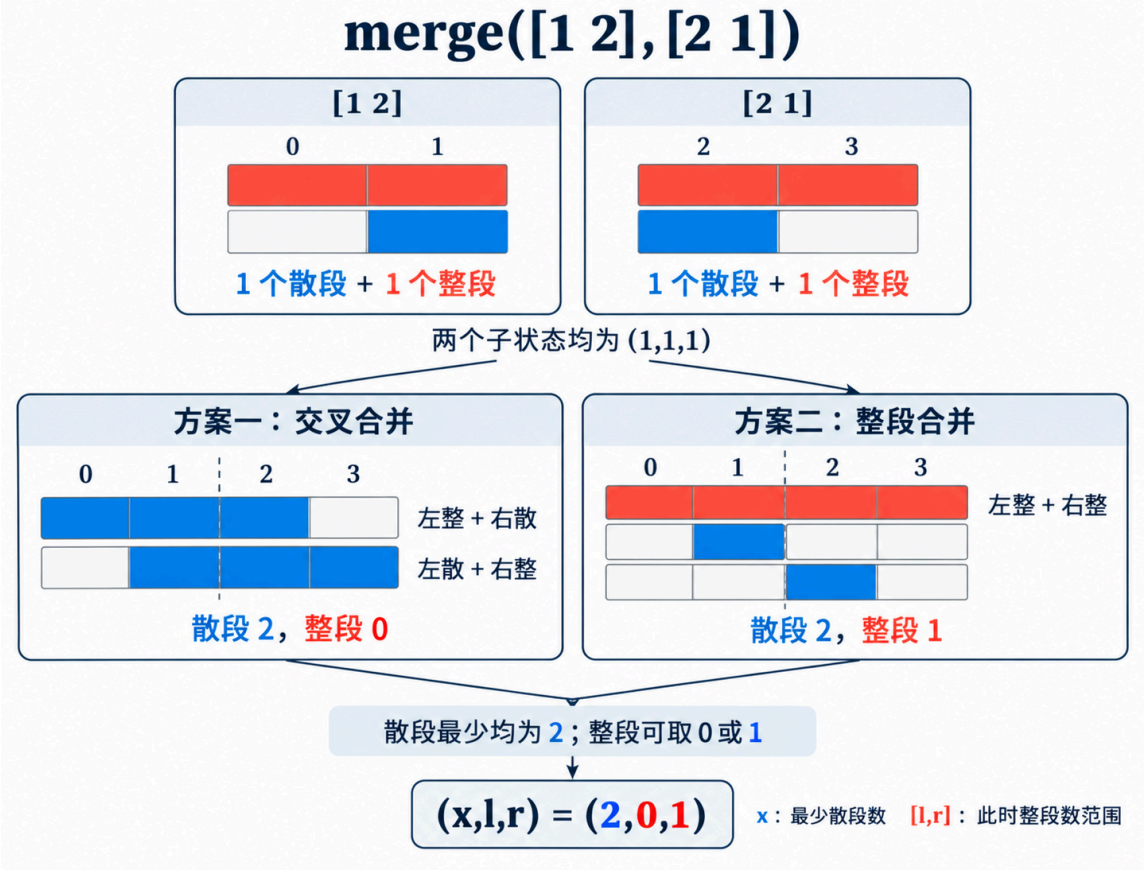


SOLUTION

Gungnir Fracture

接下来考虑优化状态空间。我们发现，用整段换一个散段肯定不劣，删除一个散段也一定不劣（注意删除一个整段可能会变劣，例如最后一组样例的情况）。

通过这个性质，我们发现合并时，我们一定优先进行整段的合并以最小化散段个数。在散段个数达成最小后，再考虑整段是否需要自行合并来增加整段个数。此时可以进行分治过程，子结构为（最少需要 x 个散段，在此基础上有 $[l, r]$ 个整段），可以证明该结构是封闭的，时间复杂度 $O(n)$。



PROBLEM

Don't Fight The Music

给定两个序列 A, B，每个位置可以为 $0, 1, ?$。

两个序列分别是对排列 P 的约束：

- A_i 的含义是 $P[1, i]$ 是否是连续区间。
- B_i 的含义是 $P[i, n]$ 是否是连续区间。

其中 1 代表必须，0 代表必须不，$?$ 代表无限制。

构造一个满足约束的排列 P，或者报告不存在。

Constraints

$|A| \leq 2 \cdot 10^5$。

Idea by **dXqwq** & **Gemini 3.1 Pro**

Prepared by **dXqwq**

8 Teams Attempted

2 Teams Solved

First Blood: 梦熊-PKU-1 @ 144 min

Expected Difficulty: Advanced

Judge Solution: 1592 Bytes

本题是 And Revive The Melody 的前传。

SOLUTION

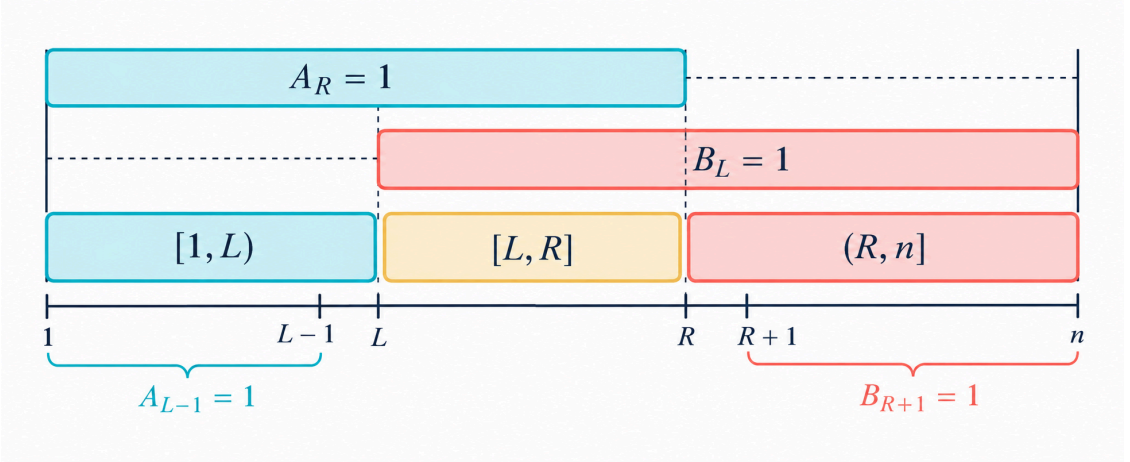
Don't Fight The Music

设 $1 < L \le R < n$，且 $A_R = B_L = 1$，即两个真前后缀同时连续。

两个值域都是连续区间，交集非空、并集为 $[1, n]$，且各有对方不含的元素。因此它们必从值域两端相交。

不妨将所有值整体翻转，使 $P[1, R]$ 的值域为 $[1, R]$，$P[L, n]$ 的值域为 $[L, n]$。

于是三个位置段 $[1, L)$、$[L, R]$、$(R, n]$，恰好分别使用对应的三个连续值域。



结论：两个外侧剩余部分也必须连续。

$$A_R = B_L = 1 \Rightarrow A_{L-1} = B_{R+1} = 1$$

SOLUTION

Don't Fight The Music

先将 A_1, A_n, B_1, B_n 固定为 1；若其中有 0，则无解。

将 B 平移一位，记切口类型 $c_i = (A_i, B_{i+1})$，其中 $1 \le i < n$。

对确定的 01 约束，充要条件是：

- 忽略 00 后，依次为若干个 10、若干个 11、若干个 01。
- 若没有 11，最后一个 10 与第一个 01 之间，必须至少有一个 00。

例如：$10, 01$ 不可行，$10, 00, 01$ 可行。

若存在 11，记最前、最后的 11 切口为 x, y。

左侧 $[1, x]$ 使用 $1 \sim x$，在每个 $A_i = 1$ 的位置之后分块；各块值域从大到小，**块内升序**。

相邻两个 11 切口 $l < r$ 之间，在位置 $[l+1, r]$ 填入

$$r, r-1, ..., l+1$$

右侧 $[y+1, n]$ 使用 $y+1 \sim n$，在每个 $B_i = 1$ 的位置之前分块；同样让各块值域从大到小，**块内升序**。

首项为 x、末项为 $y+1$，分别阻止左侧后缀、右侧前缀提前连续。

SOLUTION

Don't Fight The Music

若不存在 11，记最后一个 10、第一个 01 的切口分别为 x, y。

端点保证二者存在，且 $y \geq x + 2$。令

$$P = (U, n, x + 2, ..., y - 1, 1, V)$$

U, V 分别使用值域 $[2, x + 1]$、$[y, n - 1]$，满足左侧 A、右侧 B；中间递增段可为空。

对 $x < i < y$，前缀含 $2, n$，后缀含 $1, n - 1$，但各自长度不足 $n - 1$，所以两边都不连续。

构造 U 时，在每个 $A_i = 1$ 之后分块；构造 V 时，在每个 $B_i = 1$ 之前分块。

这次各块分配**从小到大**的连续值域，**块内降序**。每块接完时恰好连续，块内未接完时则有缺口。

例如切口 $10, 00, 01$ 可以构造为

$$P = (2, 4, 1, 3)$$

若 $y = x + 1$，两端区间之间只剩一个元素；将它接到左边或右边，至少一边仍连续，所以无法同时满足两个 0。

SOLUTION

Don't Fight The Music

有 ? 时，用四个状态做 DP。

每个切口只选择与原字符兼容的类型，按从左到右的顺序处理。

- 状态 0：尚无 11，末尾为 10。
- 状态 1：尚无 11，最近的 10 后已经出现 00。
- 状态 2：已经出现 11，尚未进入 01 区域。
- 状态 3：已经进入 01 区域。

初始为空串，放在状态 0；四个端点仍先按前面的要求固定为 1。

行为当前状态，列为新切口类型。转移如下，横线表示禁止：

状态	10	00	11	01
0	0	1	2	—
1	0	1	2	3
2	—	2	2	3
3	—	3	—	3

最后只接受状态 2 或 3。记录前驱还原切口类型，再按前两页构造排列。

每个位置只有常数种转移，总时间、空间复杂度均为 $O(n)$。

PROBLEM

Lucid Traveler

有一个仙人掌，你知道其点数 n，但不知道其结构。

你可以询问若干次 (x, y, S)，其中 x, y 都是点，S 是点集，交互库会返回删除 S 中所有点后，(x, y) 在图上是否连通。

在 $3n^2$ 次询问，且 $\sum(|S| - 1) \leq n\lceil \log n \rceil$ 的限制内求出仙人掌的结构。

Constraints

$n \leq 256$。

Idea by **MnZn**

Prepared by **luqyou**

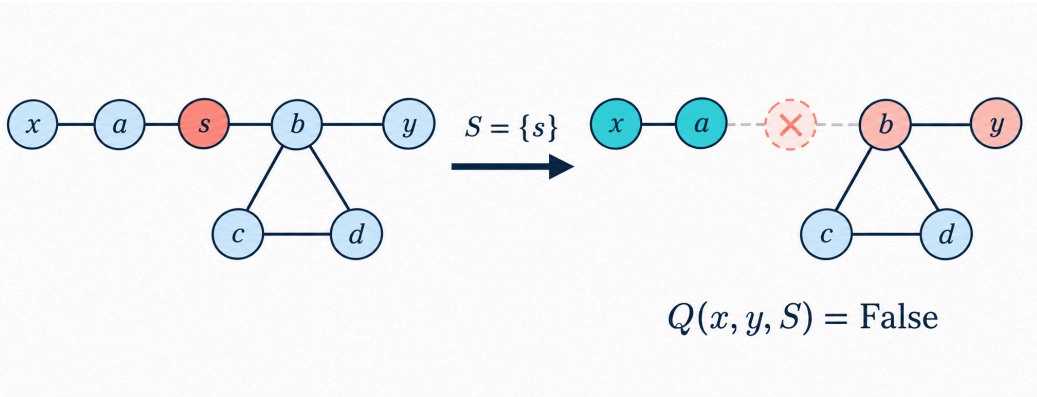
6 Teams Attempted

5 Teams Solved

First Blood: UTG @ 130 min

Expected Difficulty: Advanced

Judge Solution: 1362 Bytes



SOLUTION

Lucid Traveler

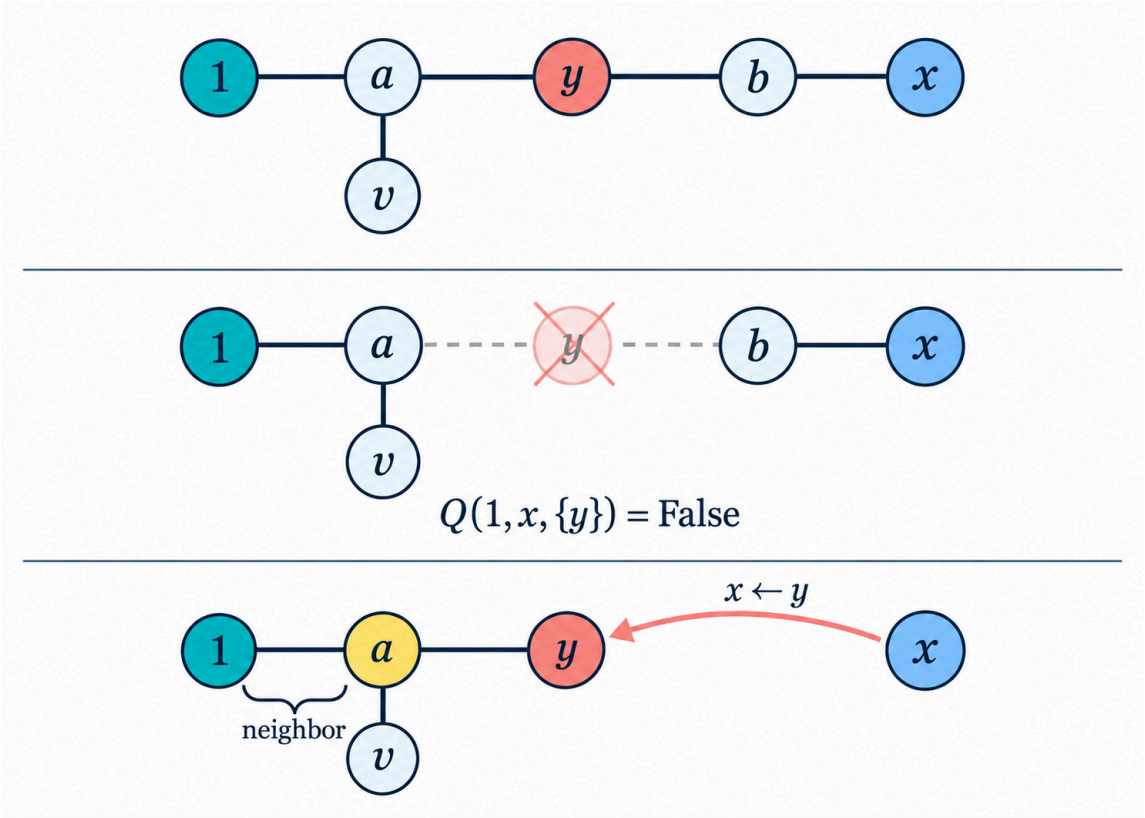
一棵树怎么做?

一个简单的想法是对于一个点不断扩展相邻的边。

考虑如何找到和 1 相邻的边，有如下算法：

- 初始化任意一个不为 1 的点 x。
- 对于剩下的点 y，依次询问 $Q(1, x, \{y\})$，如果答案为否，则令 $x \leftarrow y$。

考虑如果 $Q(1, x, \{y\}) = \text{False}$，则 y 一定在 $(1, x)$ 的路径上，而初始路径上距离 1 最近的点一定会在如上过程中胜出。



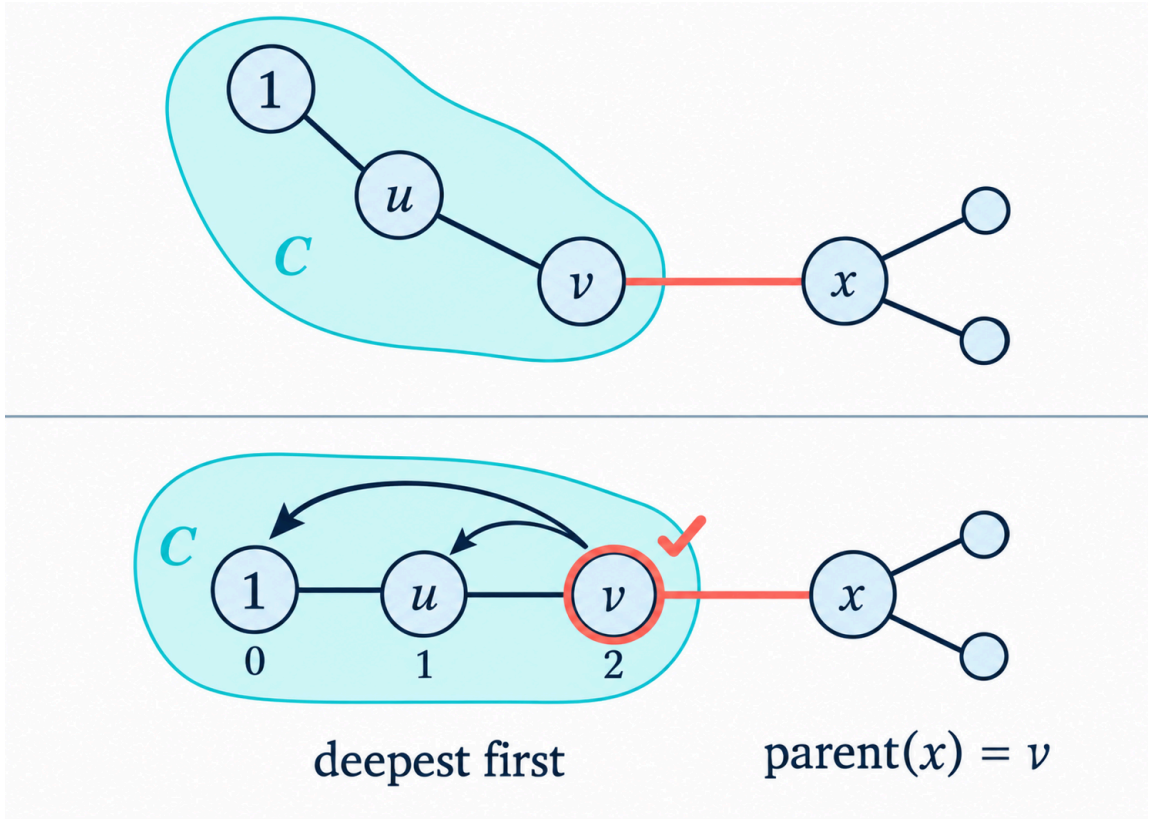
SOLUTION

Lucid Traveler

将 1 换为一个包含为 1 的连通块不会改变该算法的有效性，因此可以将前一轮找到的点在后续删除。

此时得到相邻的点 x 之后还需要考虑是与连通块中的哪个点相邻，这里的做法有很多，一种相对好些的做法是按照深度排序后从大到小枚举所有连通块内的结点，第一个在 $(1, x)$ 路径上的就是答案。

综上，我们可以用 n^2 次询问，且所有询问均满足 $|S| = 1$ 的基础上得到树的结构。



SOLUTION

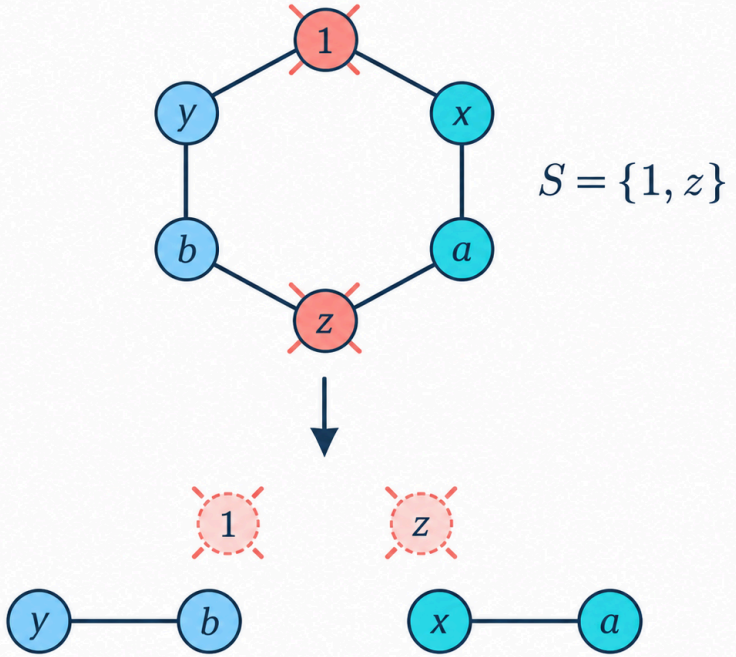
Lucid Traveler

一个环怎么做?

由于环是循环同构的，我们可以固定一个点之后对剩余的点进行排序。

对于环上的问题，显然 $|S| > 1$ 的询问才有意义。我们考虑固定 1 之后让 $S = \{1, z\}$，那么此时询问的内容等价于 z 是否在 (x, y) 中间。

可以运行插入排序：假设我们已经知道了前 k 个点组成的环，则可以询问点是否在一段区间内。这种方法可以二分，唯一的问题是无法判断第一段和最后一段的情况，需要单独处理。总询问次数仍然不超过 $n\lceil \log n \rceil$，且每次询问都有 $|S| = 2$。



SOLUTION

Lucid Traveler

整个题怎么做?

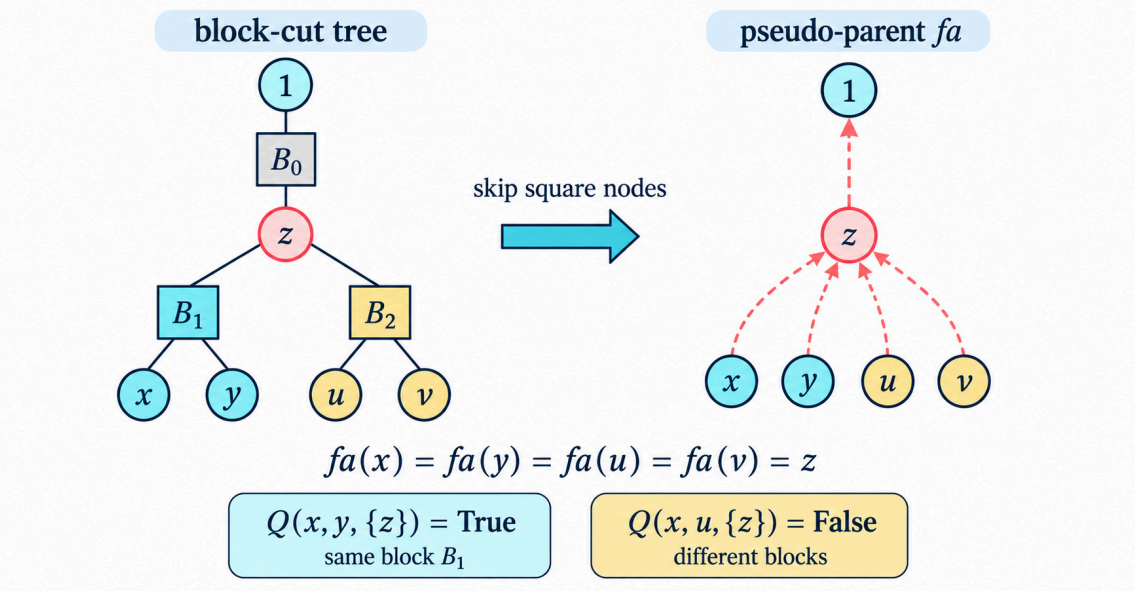
我们建立**圆方树**，第一部分树上算法的询问仍然可以找到圆方树上最接近 1 的圆点。

对于同父亲 z 的圆点 (x, y)，区分其方点是否相同可以只依靠 $Q(x, y, \{z\})$，因此我们同样可以在 $O(n^2)$ 次询问内重建完整的圆方树。

在此之后，只需要使用处理环的算法对每个环求排列即可。

总询问次数 $(2 + o(1))n^2$，代价不超过 $n\lceil \log n \rceil$。

Fun fact: 在手写次数为 $n\lceil \log n \rceil$ 的同时，`std::stable_sort` 最坏可能达到 $n(\lceil \log n \rceil + 2)$，而 `std::sort` 甚至不能在 $n(\lceil \log n \rceil + 2)$ 次之内完成。



PROBLEM

Incyde

有一个长度为 n 的环 $a_0, a_1, ..., a_{n-1}$，满足 $a_i = i$。

考虑 $\binom{n}{m}$ 种将环切为 m 段的方案：对于一种方案，其权值为每一段的**和**的最大公因数。

求出所有方案的权值**积**，答案对 $10^9 + 123$ 取模。

Constraints

$T \leq 100, n \leq 10^{12}$。

Idea by **orangejuice**

Prepared by **dXqwq**

0 Teams Attempted

0 Teams Solved

First Blood: Not Available

Expected Difficulty: Advanced

Judge Solution: 6812 Bytes

SOLUTION

Incyde

考虑拆质因子的贡献。记 E_q 为**每段的和都被 q 整除**的划分方案数，则

$$\mathrm{Ans} = \prod_{p \text{ 为质数}} p^{\sum_{k\geq 1} E_{p^k}}$$

记总和 $S = \frac{n(n-1)}{2}$。只有 $q = p^k \mid S$ 时才可能有贡献，这样的质数幂只有 $O(\log n)$ 个。

分别分解 n 和 $n-1$，再将 2 的指数减一，即可得到 S 的分解，避免直接计算乘积溢出。

固定 $q \mid S$。在 a_i 前切开，对应前缀和 $s_i = \frac{i(i-1)}{2}$，其中 $0 \leq i < n$。

每段和被 q 整除，当且仅当选中的 m 个前缀和**模 q 相同**。跨过环首的那段也由 $q \mid S$ 保证。

令 $c_r = \#\{i : s_i \equiv r \bmod q\}$，则

$$E_q = \sum_{r=0}^{q-1} \binom{c_r}{m}$$

接下来只需统计**不同桶大小及其数量**，无需枚举 q 个余数。

SOLUTION

Incyde

先考虑奇质数 p，令 $q = p^k$。

$$8s_i + 1 = (2i-1)^2$$

s_i 模 q 以 q 为周期；桶大小等于对应平方余数的根数。

非零平方余数的 p 因子次数为 $2t < k$。约去 p^{2t} 后有两个单位根，各有 p^t 种提升。

该类共有 $(p-1)\frac{p^{k-2t-1}}{2}$ 个桶，每桶有 $2p^t$ 个位置。

平方余数 0 单独一桶，有 $p^{\lfloor \frac{k}{2} \rfloor}$ 个位置。

由 $q \mid S$，写成 $n = hq + e$，其中 $e \in \{0, 1\}$。完整周期的贡献为

$$E_q = \binom{hp^{\lfloor \frac{k}{2} \rfloor}}{m} + \sum_{0 \le 2t < k} \frac{(p-1)p^{k-2t-1}}{2} \binom{2hp^t}{m}$$

若 $e = 1$，$s_i = 0$ 的桶（平方余数 1）多一个位置，再补上

$$\binom{2h+1}{m} - \binom{2h}{m}$$

SOLUTION

Incyde

再考虑 $q = 2^k$。

此时周期为 $2q$，每个余数在一个周期内恰好出现两次：对应的位置满足 $j \equiv i$ 或 $1 - i \bmod 2q$。

由 $q \mid S$，写成 $n = 2hq + e$，其中 $e \in \{0, 1\}$。因此

$$E_q = (q - e)\binom{2h}{m} + e\binom{2h+1}{m}$$

每个 p^k 只处理 $O(k)$ 类桶，所有质数幂合计只需求 $O(\log^2 n)$ 次组合数。

每次组合数计算为 $O(\log n)$；分解 n、$n-1$ 只需试除不超过 10^6 的质数。

令模数 $P = 10^9 + 123$。当 $p \neq P$ 时，指数只需对 $P - 1$ 取模，而

$$P - 1 = 2 \cdot 3 \cdot 11 \cdot 2089 \cdot 7253$$

对这五个小质数分别用 **Lucas 定理**计算组合数，再用 **CRT** 合并，最后快速幂相乘。

若 $p = P$，必须单独判断指数是否为正。只需看模 P 的最大桶是否能选出 m 个位置：

$$m \le 2\left\lfloor \frac{n}{P} \right\rfloor + (n \bmod P)$$

成立则答案为 0，否则该质因子无贡献。

PROBLEM

False Amber

两个人各有 n 张牌，并从自己的牌里随机抽取一张。先手做一次决策，可以选择 Check 或者 Allin。如果先手 Allin，后手可以选择 Fold 或者 Call。游戏结果如下：

- 先手 Check：赢家 $+1$，输家 $+0$。
- 后手 Fold：先手 $+1$，后手 $+0$。
- 后手 Call：赢家 $+(m+1)$，输家 $-m$。

先手需要制定一个策略，并将其告诉后手，问双方最优策略下先手的期望收益，输出浮点数。

Constraints

$n \leq 2 \cdot 10^5$。

Idea by **dXqwq**

Prepared by **dXqwq**

2 Teams Attempted

1 Teams Solved

First Blood: 击中月亮 @ 192 min

Expected Difficulty: Advanced

Judge Solution: 926 Bytes

~~太长不看：两个人在打牌，给定 SPR 和范围，问 River IP Allin Spot 的双方 EV。~~ 事实上题目和上述描述仍然存在一定区别，EV 的计算是纳什均衡，因此后手不是纯策略。

SOLUTION

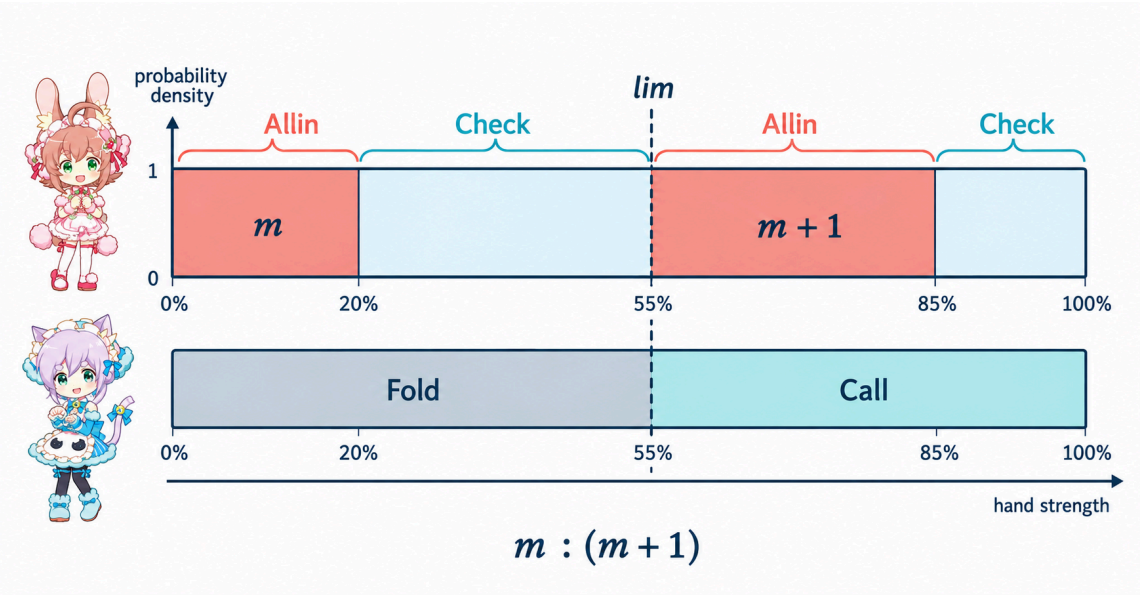
False Amber

我们先假设双方的牌力都可以视为一个概率密度函数（PDF），以简化下方对阈值的定义。

对于一组先手的策略，后手的每一张牌，选择 Call 和 Fold 都会有固定的收益，并且该收益随着手牌强度单调，因此后手会 Call 一个后缀。

我们不妨思考，先手在确定希望后手 Call 的一个后缀 $[\lim, +\infty)$ 之后，该如何制定策略。为了使该阈值成立，先手 Allin 的 PDF 中 $< \lim$ 的部分和 $> \lim$ 的部分应该为 $m : m+1$。

之后，由于两部分中每一手牌 Allin 和 Check 的收益都已固定，我们显然会选择两部分中各自较差的一个前缀。



SOLUTION

False Amber

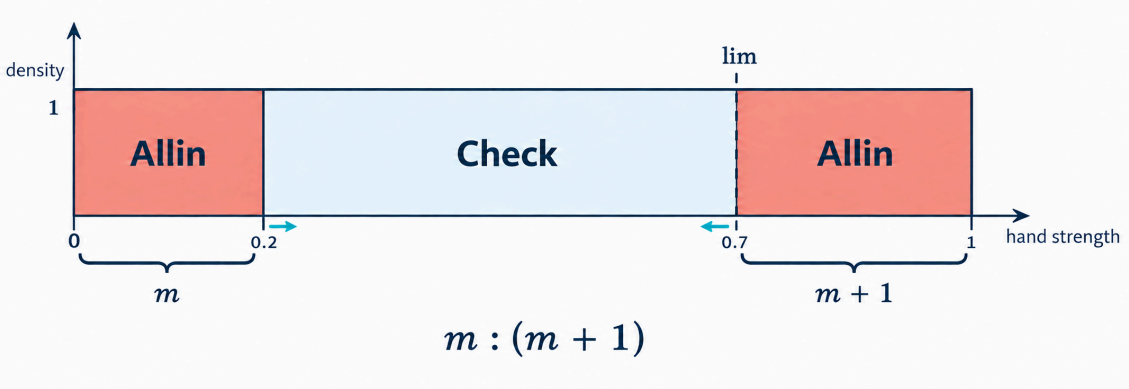
但我们此时考虑，如果将 $> \lim$ 部分的前缀进行微扰，可以得到该策略对该前缀的导数，可以分两种情况：

- 导数为正：更大的牌会让 $\lim$ 变小，并让导数变得更大，此时我们取满剩余所有 $> \lim$ 的牌显然会更优。
- 导数为负：此时所有 Allin 手牌的收益显然都比 Check 更小，不如都不 Allin，全部 Check。

因此，我们会发现，最优解只能形如如下结构：

- Allin 了手牌的一个前缀和一个后缀，且前后缀比例恰好为 $m : (m+1)$。

最优解显然会在 PDF 的某个拐点取到，双指针枚举所有情况即可在 $O(n)$ 的时间复杂度内求解。



PROBLEM

Break the Speakers

有一棵树，树上的每个点有权值 a_i。你可以做以下三种操作：

- 选择一个 $a_i = 0$ 的叶子并删除。
- 选择一个 $a_i > 0$ 的点并令 $a_i \leftarrow a_i - 1$
- 选择一个 a_i 修改为其所有邻居 a_j 的 MEX。

求出删空树的操作次数最小值。

Constraints

$n \leq 2 \cdot 10^5$。

Idea by **luqyou**

Prepared by **luqyou**

37 Teams Attempted

0 Teams Solved

First Blood: Not Available

Expected Difficulty: Expert

Judge Solution: 1397 Bytes (luqyou)

SOLUTION

Break the Speakers

删除每个点的操作固定贡献 n，下面只计算修改点权的次数。

固定最后删除的点作为根。父节点一定比子节点晚删除，但**可以先改变父节点的权值**，帮助子树清空。

设 $f_u(t)$ 为父节点权值固定在状态 t 时，清空 u 的子树所需的最少修改次数。

三种边界状态分别为 $t = 0, 1, 2$，表示父节点权值为 0、1、≥ 2。

同一父节点状态下，对不同子树的操作可以集中处理。只需考虑：

- $a_u > 0$：处理部分儿子，再将 u 清零，处理剩余儿子。
- $a_u = 0$：始终保持零，或经历一次 $0 \to$ **正数** $\to 0$。

儿子清零后**不必立即删除**：保留这个零点，可以让父节点的 MEX 变为正数；等父节点改变后再删除它。

这也是转移时必须允许“清零但暂留”的原因。

SOLUTION

Break the Speakers

若 $a_u = 1$，只需一次减一。每棵子树可以在 u 清零前或清零后处理：

$$f_u(t) = 1 + \sum_v \min(f_v(1), f_v(0))$$

若 $a_u \geq 2$，令 W 为提前处理全部儿子的代价，D 为推迟部分儿子的节省：

$$W = \sum_v f_v(2)$$

$$D = \sum_v \min(0, f_v(0) - f_v(2))$$

对于 $a_u \geq 2$，转移为

$$f_u(0) = W + D + 2$$

$$f_u(1) = W + \min(1, D + \min(a_u, 3))$$

$$f_u(2) = W + \min(1, D + 2)$$

全部儿子删除后，父节点非零时一次 MEX 即可清零。

值得推迟的儿子必有 $a_v = 0$。保留这些零点时，父节点为 0 或 ≥ 2，可 MEX 到 1 再减一；父节点为 1，则 MEX 到 2 再减两次，也可直接减 a_u 次。

SOLUTION

Break the Speakers

若 $a_u = 0$，始终保持零的代价为 $Z = \sum_v f_v(0)$。

否则考虑一次 MEX：将 u 变为 1 或 ≥ 2，分别记为状态 $b = 1, 2$。

用两位掩码 M 记录此时邻居中是否出现 0 和 1，其余权值不影响 MEX 属于哪一类。

每个儿子选择：提前清空并暂留零点，花费 $f_v(0)$；或保留初值 a_v，之后花费 $f_v(b)$ 清空。

对每个 b 做四状态 DP，初始 $g_b(0) = 0$，其余为 $+\infty$。记 $\beta(x)$ 在 $x = 0, 1, \geq 2$ 时分别为 $1, 2, 0$。

每合并一个儿子 v，枚举旧掩码 M，用以下候选更新新状态的最小值：

$$g'_b(M \text{ or } 1) \leftarrow g_b(M) + f_v(0)$$

$$g'_b(M \text{ or } \beta(a_v)) \leftarrow g_b(M) + f_v(b)$$

合并父节点的掩码 $\beta(t)$ 后，恰为 1 才能 MEX 到 1；恰为 3 才能 MEX 到 ≥ 2。

SOLUTION

Break the Speakers

记上页两类合法掩码的最小代价为 $H_1(t)$、$H_2(t)$，不可达时为 $+\infty$。

$f_u(t)$ 是 Z 与下列两者的最小值：

$$H_1(t)+2,\quad H_2(t)+2+[t=0]$$

变为 1：一次 MEX，再一次减一。

变为 ≥ 2：一次 MEX；儿子删完后，父节点非零时再一次 MEX 清零，父节点为零时则需 MEX 到 1 后再减一。

根没有父节点，可按状态 $t=2$ 计算。

最后用**换根 DP** 枚举最后删除的点。每条有向边维护一个三元组 $(f(0), f(1), f(2))$。

先自底向上求儿子到父亲的消息，再自顶向下补齐反方向。

求和项直接减去当前儿子；四状态 DP 用前后缀合并，排除一个儿子的代价为 $O(1)$。

将所有邻居合并到 r 后，答案为

$$n+\min_r f_r(2)$$

总时间、空间复杂度均为 $O(n)$。

PROBLEM

Kikai No Tsubasa

这是一道通信题。

给定一个图，需要给点赋值点权排列 p，使得所有边 (u, v) 都满足 $p_u + p_v \leq n + 1$。

第一次运行会拿到一个解，需要用 $5 \cdot 10^5$ 个 Bit 的信息将它传给第二次运行来构造一组解，第二次运行构造的解和第一次运行不需要一样。

Constraints

$n \leq 10^5, m \leq 5 \cdot 10^5$。

Idea by **Coffee_zzz**

Prepared by **dXqwq**

18 Teams Attempted

2 Teams Solved

First Blood: UTG @ 99 min

Expected Difficulty: Expert

Judge Solution: 994 Bytes

SOLUTION

Kikai No Tsubasa

先考虑在传输 0 bit 信息时可能遇到的问题。

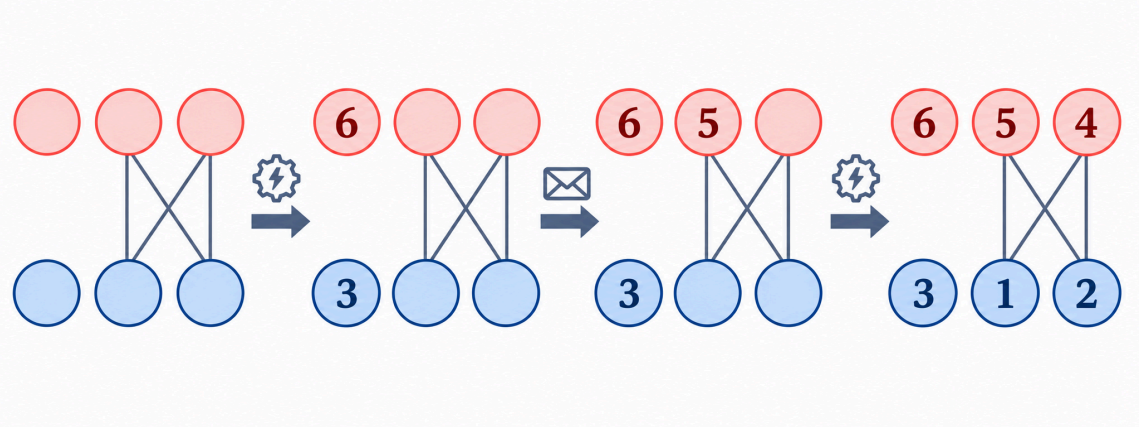
一个朴素的贪心是找一个可以放 n 的位置，这个点的度数一定 ≤ 1，在它的邻边放 1，最后删除这两个点，迭代 $\frac{n}{2}$ 次，可以解决树上的情况。

问题在于，图不总是连通的，一些孤立点加一张稠密图可能会让上方的做法找不到度数 ≤ 1 的点。

解决这个问题并不难：我们在不存在这样的点的时候可以直接从第一次运行"索取"一个最大的数。考虑进行"索取"可以确定至少 2 个小点和 2 个大点，因此只需要传输 $\frac{n}{4}$ 个数，即 $\frac{n}{4}\lceil \log_2 n \rceil$ 比特。

索取的过程可以视作交互题：第一次运行可以模拟第二次运行的贪心，将卡住时需要的点预先编码进消息。

为便于证明，可先传 n 位大小点标记；关键点用大点集合内的下标表示，每项 16 位。



PROBLEM

And Revive The Melody

给定两个序列 A, B，每个位置可以为 $0, 1, ?$。

两个序列分别是对排列 P 的约束：

- A_i 的含义是 $P[1, i]$ 是否是连续区间。
- B_i 的含义是 $P[i, n]$ 是否是连续区间。

其中 1 代表必须，0 代表必须不，$?$ 代表无限制。

求出满足约束的排列 P 数量模 998244353 的值。

Constraints

$|A| \leq 2 \cdot 10^5$。

Idea by **dXqwq** & **Gemini 3.1 Pro**

Prepared by **dXqwq**

0 Teams Attempted

0 Teams Solved

First Blood: Not Available

Expected Difficulty: Master

Judge Solution: 3784 Bytes

本题是 Don't Fight The Music 的续作。

SOLUTION

And Revive The Melody

考虑 Don't Fight The Music 中的结构为什么不能支持计数：形如 $00\cdots01$ 的序列要求任意子区间都不是前缀，这个限制不是很好计数。

因此，我们考虑容斥掉所有的 0，先考虑只有 1 和 $?$ 的子问题。和 Don't Fight The Music 类似地，我们进行同样的观察。

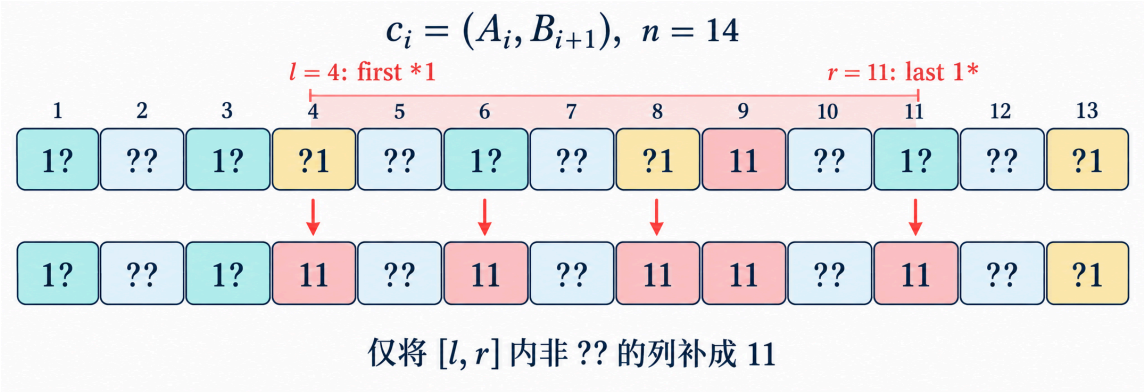
- 不存在 $i \geq j$ 的 $A_i = B_j = 1$

这种情况下贡献 A_i 和 B_j 的区间是不交的，只需要先求一个阶乘进行两段区间的划分，然后递归单侧的子问题即可。

单侧的子问题显然是显著简单的，记 $S = \{i | A_i = 1\}$，则从大往小考虑所有约束，答案即为 $\prod \left(S_{\{i+1\}} - S_i + 1\right)!$。

- 存在 $i \geq j$ 的 $A_i = B_j = 1$

用 $A_i = B_{i+1} = 1$ 作为分割后可以规约到前者。



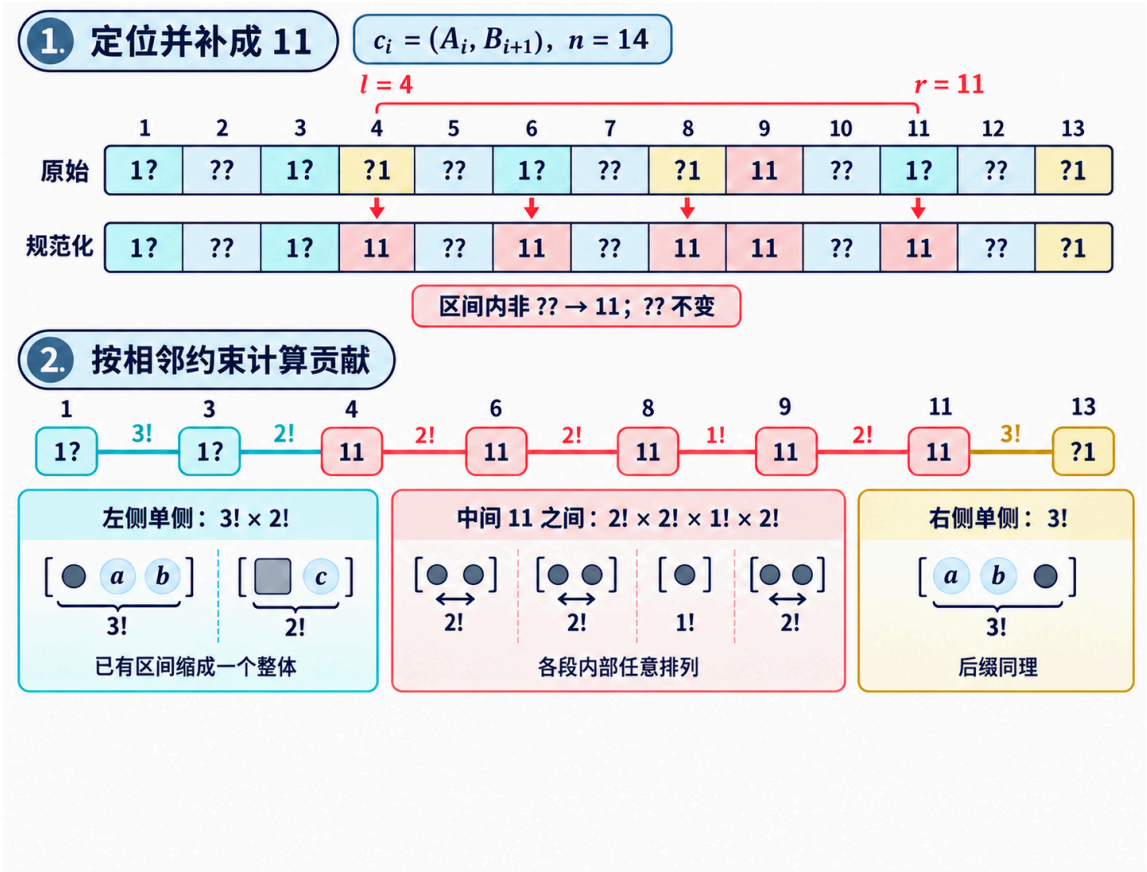
SOLUTION

And Revive The Melody

接下来我们只需要使用容斥原理，将 0 视为 $?$ 的方案数与 1 的方案数的差值即可。对于一个没有 0 的序列，我们有如上所述的算法，接下来只需要通过这个过程来动态决定每个 0 是取 1，提供 -1 的贡献还是取 $?$，提供 1 的贡献。

直接做 DP 求解的复杂度是 $O(n^2)$ 的，但注意到转移只和两个有 1 列之间的 $(0,0)$ 列数量有关，可以使用分治 NTT 优化 $i \to j$ 的转移系数只和 $(j-i)$ 有关的 DP，总时间复杂度 $O(n \log^2 n)$。

本题实现细节可能较多，需要谨慎规划分类讨论与边界情况的系数计算。



PROBLEM

Ether Second

给定一个序列 a，满足 $a_i \in \mathbb{F}_2^{64}$。

你需要维护以下三种操作：

- 对于 $[l, r]$ 中 $\text{popcount}(a_i \oplus x) > 3$ 的所有 a_i，将其赋值为 x。
- 对于 $[l, r]$ 中的所有 a_i，将其赋值为 x。
- 求出区间 $[l, r]$ 的所有元素的和。

Constraints

$n, m \leq 2 \cdot 10^5$。

Idea by **orangejuice**

Prepared by **orangejuice** & **dXqwq**

0 Teams Attempted

0 Teams Solved

First Blood: Not Available

Expected Difficulty: Master

Judge Solution: 6890 Bytes (orangejuice)

将会讲解验题人（dXqwq）的做法，与 4 Argon 的大致相同，与 6 Astra 的有较大不同。该做法需要一定的常数优化通过此题。

SOLUTION

Ether Strike

（假的）算法 A：

- 分块
- 底层维护每个块所有出现过的数
- 对于每个数，维护其出现过的数的集合

问题：

1. 无法快速维护修改
2. 没有利用修改操作后，颜色数变小的趋势。

另一个（假的）算法 B：

- 用线段树组织这些块
- 修改时仍然暴力遍历所有块
- 每次操作完之后，如果两个叶子的颜色数之和同样小于阈值，就将子树进行合并
- 块内修改则同样可以分裂底层块

问题：

1. 全局颜色数少且局部颜色数多时表现较差
2. 对于 1 操作难以快速找到合并的元素

SOLUTION

Ether Strike

核心观察：

对于一个集合内的元素，可以分两类维护。

- 至少被一次 1 操作修改
- 维持初始值

现在，我们考虑一个大点。我们可以证明……

第二类元素的值域集合在被缩小 $K = 12$ 次之后，其大小不会超过 $M = 4$。又注意到在上述集合不变时，第一类元素的值域集合也不会增加。因此，此时全局元素数同样不会超过 $12 + 4 = 16$。

证明：`codex exec "帮我证明这个结论"`

于是我们可以将算法 B 的合并阈值设为该条件。

对于每个大点，我们都大方地考虑其对于 $2^{64} \to 2^{64}$ 的映射表中的两类集合。

假如我们每次操作都改变了第二类值域集合大小，则每个结点总势能为 K，每次操作至多增加 $\log n$ 个点的性能，即最终会对 $\frac{n}{K+M}K + q \log n$ 个小点做操作，以及进行 $n + q \log n$ 次合并和 $q \log n$ 次分裂。

对于每个小点的操作，粗略地分析复杂度，大致不会超过颜色数和对线段树的合并/分裂，后者的复杂度是整体 Bound 的，前者是 $(K + M) \log n$。

SOLUTION

Ether Strike

最后要解决两个问题：

1. 快速处理不改变第二类元素值域集合的操作

注意到在线段树上的大点部分，孩子永远比祖先被更多的数操作过，因此其对于大数的维护更准确

因此，可以对这 $\leq K$ 个点暴力维护一个映射作为其 Lazy tag，同时对于这些点维护出现次数（可以从孩子继承，孩子一定有）以同时维护懒标记导致的区间和变化。

2. 判定操作是否改变第二类元素值域集合

我们大胆猜测，在进行以下两种对称等价

- 对于每一位，0 和 1 之间翻转等价
- 对于所有位，排列交换等价

之后，有效的状态非常少，因此可以建立自动机。通过对自动机的预处理和对位运算的合理使用，我们可以在 $O(1)$ 的时间复杂度内求解子问题。

时间复杂度 $O\left(\left(n\frac{K}{K+M}+q\log n\right)(K+M)\log n\right)$。